LLMS & RAG

Notes based on LLM and RAG and theory +Project base

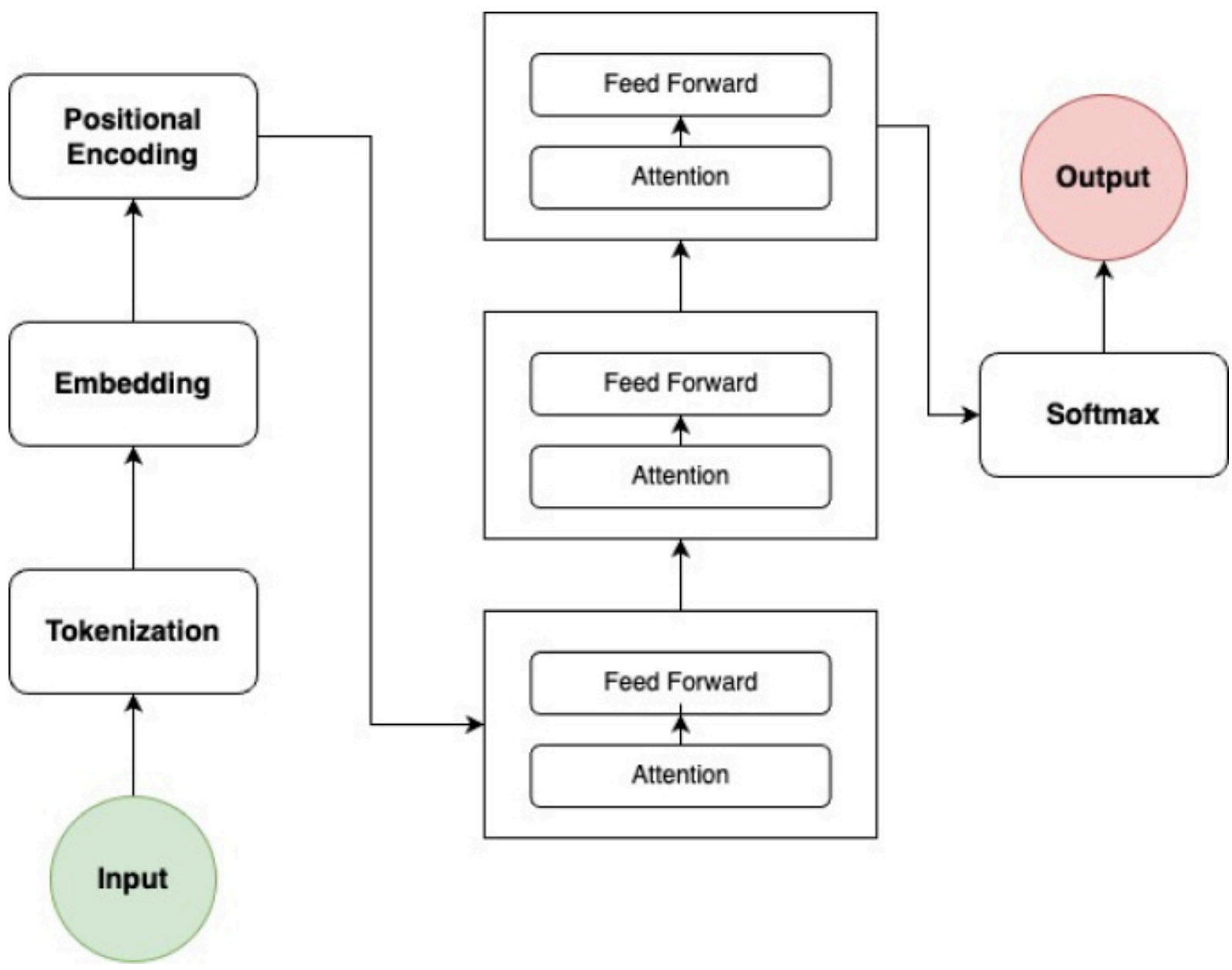
OVERVIEW

Sl.No	Title
1	Understanding the Basics
T-1	Local LLM
T-2	RAG
T-3	Connecting LLM + RAG
T-4	Minimal practical setup
T-5	Fine-tuning & enhancements

Understanding the Basics

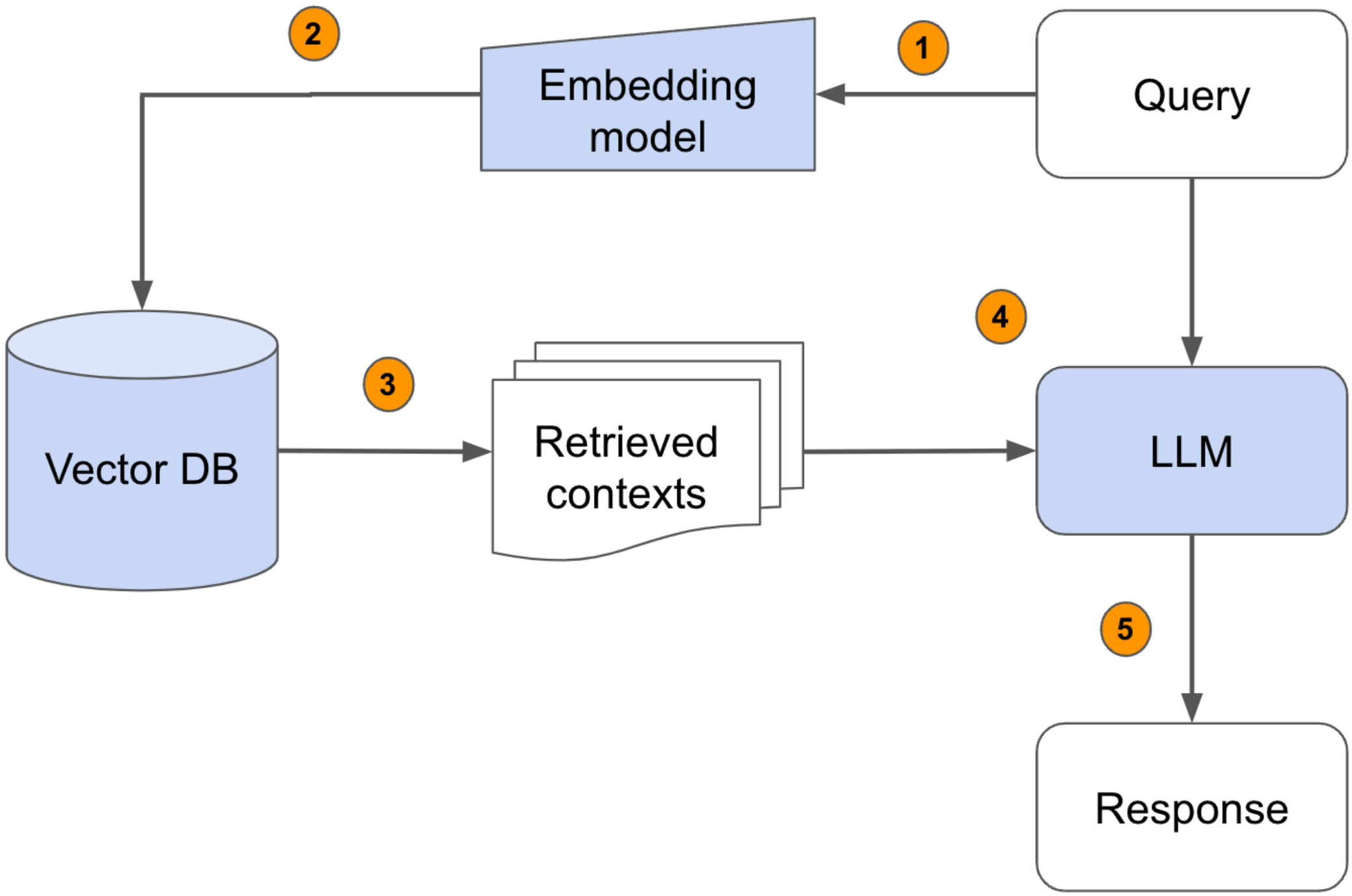
A. Local LLM

- A **Local LLM** is a large language model you run **on your machine** (CPU/GPU) instead of using online APIs like OpenAI.
- Examples:
  - **LLaMA** (Meta)
  - **MPT**
  - **Falcon**
  - **Alpaca / Vicuna** (instruction-tuned variants of LLaMA)
- Key points:
  - You need **good GPU** for large models; smaller models can run on CPU.
  - You can **fine-tune** them on your data if needed.



B. RAG (Retrieval-Augmented Generation)

- **Problem:** LLMs have knowledge cutoff. They “hallucinate” answers if info is missing.
- **Solution:** RAG augments LLMs by retrieving relevant info from your own documents/knowledge base.
- **Flow:**
  1. **Document → Split → Embed → Store in vector database**
  2. **User query → Retrieve top relevant docs → LLM generates answer using those docs**
- Tools commonly used:
  - **Text Splitter:** langchain.text_splitter
  - **Embeddings:** OpenAI Embeddings / HuggingFace SentenceTransformers
  - **Vector Database:** FAISS, Chroma, Milvus
  - **Chain:** LangChain or LlamaIndex



Theory 1: Local LLMs (Large Language Models)

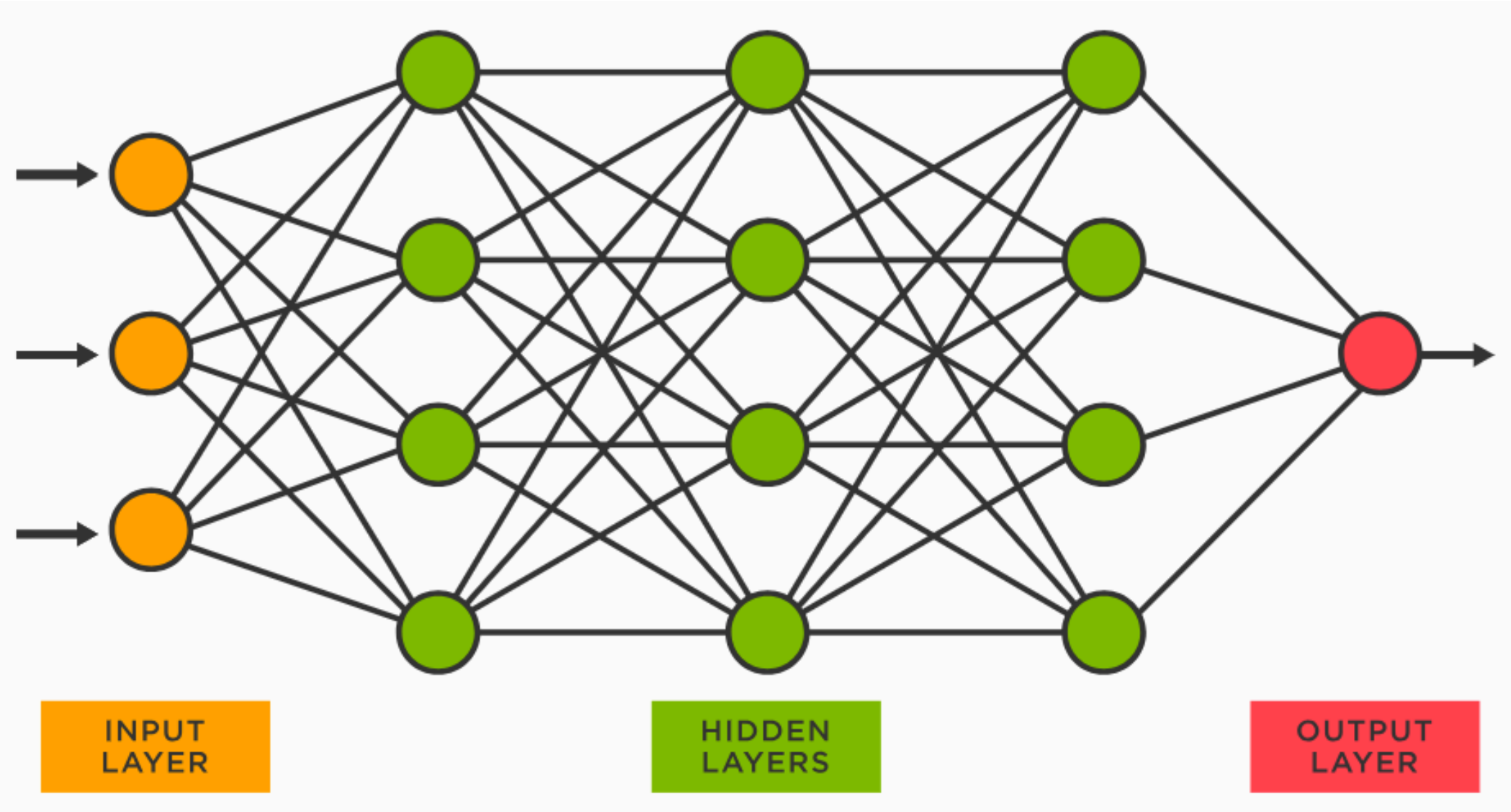
What is an LLM?

- **Definition:**
  A Large Language Model (LLM) is a **deep learning model** trained on **huge amounts of text data** to understand and generate human-like text.
- **Purpose:**
  - Answer questions
  - Generate text/code
  - Summarize, translate, chat
- **Key concept:**
  LLM predicts the **next word** in a sentence based on the previous words.

Example:

Input: "The capital of France is"

Model prediction: "Paris"

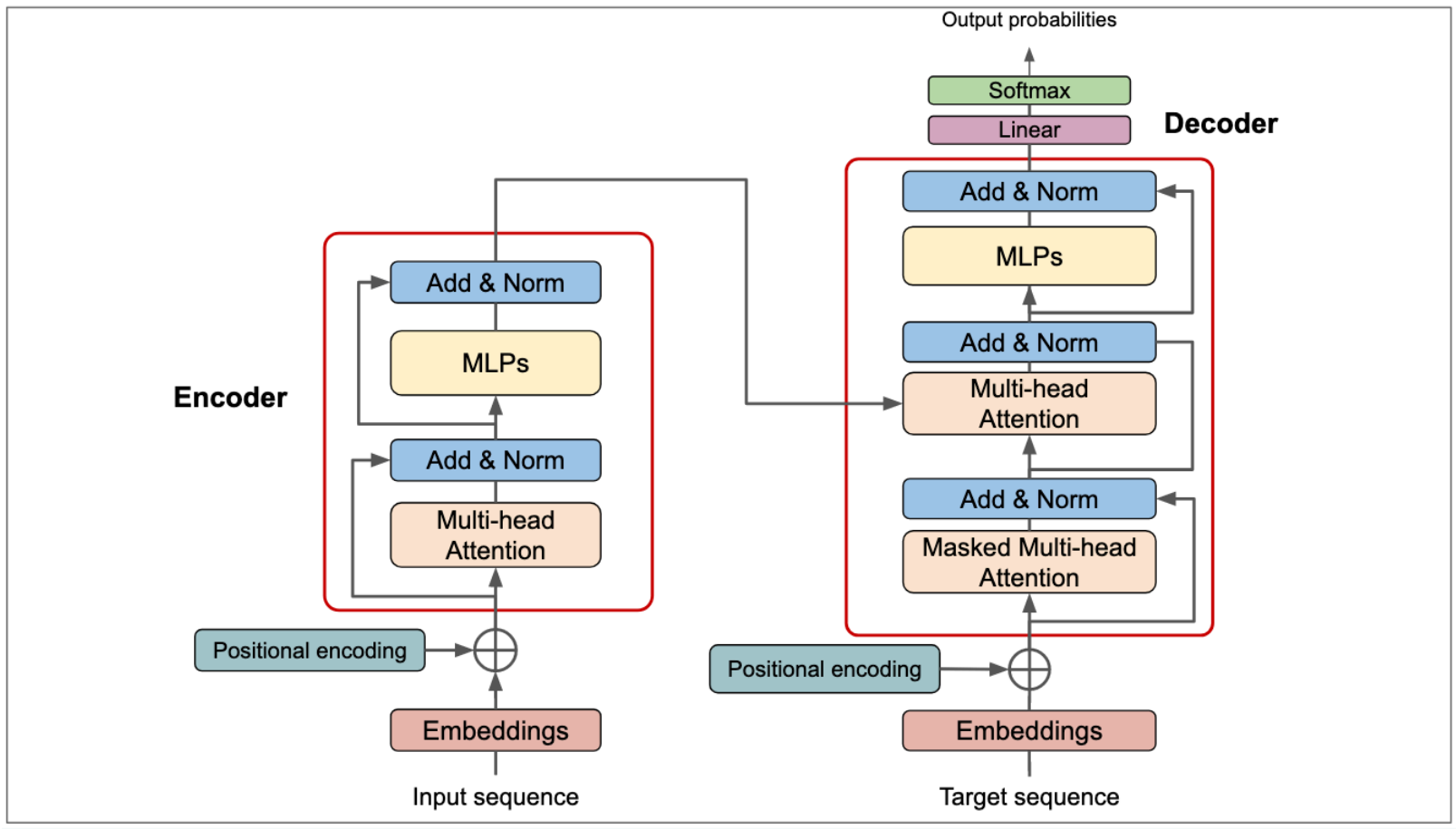


2 How LLMs Work

LLMs are based on Transformers (Vaswani et al., 2017).

Key Components:

1. **Tokenization**
   - Breaks text into smaller units (words/subwords/tokens)
   - Example: "I love AI" → ["I", "love", "A", "##I"]
2. **Embedding**
   - Converts tokens into vectors (numbers)
   - Each token gets a unique vector representing meaning
3. **Attention**
   - Transformer uses **self-attention** to understand **relationships between words**
   - E.g., "I went to the bank to withdraw money" → attention distinguishes financial vs river bank
4. **Layers**
   - Multiple transformer layers process the embeddings
   - Bigger LLM → more layers → better reasoning
5. **Decoding**
   - The model generates text using strategies like:
     - Greedy
     - Beam search
     - Sampling / Temperature



Local vs Cloud LLM

Feature	Local LLM (On-Premise/Self-Hosted)	Cloud LLM (API/Managed Services)
Data Privacy	**High** (Data never leaves your system)	**Low/Moderate** (Data sent to third party)
Costs	**High upfront hardware (GPU, RAM)**	**High recurring subscription/usage fees**
Setup/Maintenance	**High technical expertise required**	**Low; Ready-to-use, managed by provider**
Performance	**Limited by local hardware**	**Excellent; Access to top-tier models**
Internet Access	**Not required** (Works offline)	**Required** (Dependent on internet)
Customization	**Full control (Fine-tuning, RAG)**	**Limited to prompt engineering/API settings**
Scalability	**Manual; Limited by local capacity**	**High; Automatically scales to demand**
Common Use Cases	**Sensitive data, offline apps**	**General purpose, rapid deployment, high complexity**

***Local LLM is ideal for private knowledge, RAG, experiments, or fine-tuning.**

Popular Local LLMs

Model	Size	Notes
LLaMA 2	7B–70B	Meta, instruction-tuned versions exist
MPT	7B–30B	Open weights, good for code/text
Falcon	7B–40B	Fast inference, HuggingFace
Vicuna	7B	Instruction-tuned LLaMA variant
Alpaca	7B	Open-source fine-tuned LLaMA

Requirements to Run Local LLM

Hardware:

- **GPU:** Strongly recommended for >7B models (NVIDIA preferred)
- **CPU only:** Possible for small models (<1B tokens or quantized versions)
- **RAM:** At least 16GB (more for bigger models)

Software:

- Python 3.10+
- PyTorch or TensorFlow
- Transformers library (pip install transformers)
- Optional: Accelerate / BitsAndBytes for memory-efficient inference

Tips for beginners:

- Start with **7B models on GPU**
- Or use **quantized models** (4-bit or 8-bit) for CPU experiments

✅ **Key Takeaways:**

1. LLMs predict the next word using transformers.
2. Local LLMs give **privacy, cost control, and flexibility**, but need **hardware**.
3. Start with **smaller models** to experiment.
4. Local LLM + RAG lets you **combine your knowledge base** with generative AI.

Theory 2: RAG (Retrieval-Augmented Generation)

RAG is a method to **combine your own knowledge/data with an LLM** so it can answer questions **accurately** even if the info is outside the LLM's pre-trained knowledge.

1 Why RAG is needed

- LLMs have **knowledge cutoff** and **hallucinate** (make up facts)
- Example:
  - Question: "What's the latest product Apple released in 2026?"
  - LLM (trained till 2023) → may guess incorrectly
- **Solution:** Give LLM access to **your own documents** → more accurate answers

2 How RAG Works (High-Level)

RAG is basically **Retrieve + Generate**:

Flow:

1. **Input:** User asks a question
2. **Retrieve:** System finds relevant documents from your knowledge base
3. **Generate:** LLM generates answer using retrieved documents

Components of RAG

A. Document Collection

- Could be PDFs, txt files, web pages, notes, etc.
- Example: Your personal notes on AI

B. Text Splitting

- Documents are split into **smaller chunks** (100–500 tokens)
- Why? LLM can only process **limited context** at a time

C. Embeddings

- Convert text chunks into **numerical vectors** representing meaning
- Tools: sentence-transformers, OpenAI embeddings
- Similarity search uses **cosine similarity** between vectors

D. Vector Store (Database)

- Store embeddings for efficient retrieval
- Tools:
  - **FAISS** (Facebook AI)
  - **Chroma**
  - **Milvus**
- When user asks a question → system searches top-k relevant chunks

E. Retriever

- Fetches the **most relevant chunks** from the vector store

F. Generator (LLM)

- Receives **retrieved chunks + question**
- Generates answer **based on the context**

RAG Pipeline Diagram

User Query

|

▼

Retrieval Module (Vector DB search)

|

▼

Top-k Relevant Docs

|

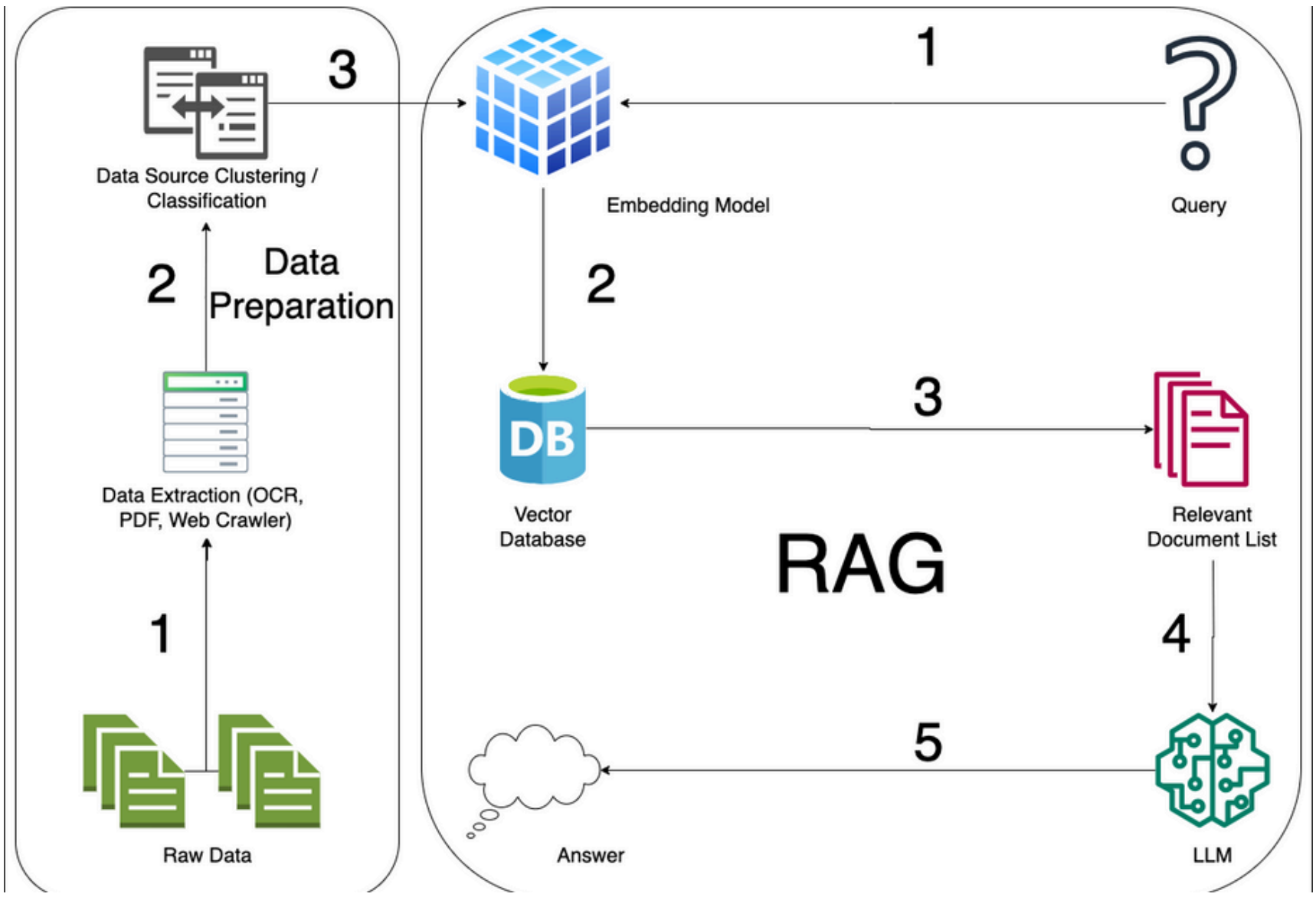
▼

LLM (Local model)

|

▼

Answer



Types of RAG

1. **Closed-book Generation**
   - LLM answers **without external info**
   - Limitation: May hallucinate
2. **Open-book Generation (RAG)**
   - LLM uses **retrieved knowledge** to answer
   - Accurate and context-aware

Why RAG + Local LLM is powerful

Feature	Benefit
Local LLM	Works offline, private, full control
RAG	Provides **up-to-date and domain-specific knowledge**
Combination	Personal Q&A bot, document search, interview prep assistant, note assistant

💡 **Example Use Case:**

- You have 5 PDFs on "Deep Learning"
- Question: "Explain attention mechanism"
- **RAG retrieves relevant sections from PDFs** → LLM generates precise answer
-

Summary / Key Takeaways

1. RAG = **Retrieve** relevant docs + **Generate** answer with LLM
2. Embeddings + Vector store are **core technology**
3. LLM alone can hallucinate; RAG improves **accuracy**
4. Works perfectly with **Local LLM** for privacy and domain knowledge

Theory 3: How Local LLM + RAG Works Together

The goal is to **use your local LLM** but make it **aware of your own knowledge base** via RAG.

Steps: **Documents → Split → Embed → Store → Retrieve → Generate**

2 Step-by-Step Flow

Step 1: Collect Documents

- Source your knowledge:
  - PDFs, text files, CSV, web pages
- Example:
  - AI notes, Python cheatsheets, interview Q&A PDFs

Step 2: Split Documents into Chunks

- Why? LLM can only process a **limited number of tokens** (~2048–4096)
- Split into **smaller logical chunks** (200–500 tokens)
- Tools:
  - langchain.text_splitter
  - nltk / custom Python scripts

Step 3: Generate Embeddings

- Convert each chunk into a **vector (numerical representation)**
- Embeddings capture **semantic meaning**
- Libraries:
  - sentence-transformers (HuggingFace)
  - OpenAI embeddings (if not fully local)
  -
- **Example:**

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

vector = model.encode("This is a chunk of text")
```

Step 4: Store Embeddings in a Vector Database

- Efficient search requires a **vector store**
- Popular options:
  - **FAISS** → lightweight, local
  - **Chroma** → Python-friendly
  - **Milvus** → large-scale
- Save embeddings + corresponding chunk text for retrieval

Step 5: Retrieval

- When the user asks a question:
  1. Embed the query into a vector
  2. Search **top-k similar chunks** in vector DB
- These chunks provide **context** for LLM

Step 6: Generation

- Pass the **retrieved chunks + user query** to the LLM
- LLM produces **accurate, context-aware answers**
- Example (pseudo-prompt):

Why This Works

- LLM alone → may hallucinate or lack recent knowledge
- RAG → **grounds LLM in your own documents**
- Local LLM + RAG = **offline, private, domain-specific assistant**
-

Optional Enhancements

Enhancement	Benefit
Top-k retrieval	Ensures only the most relevant chunks are used
Prompt templates	Better guidance to LLM
Chunk overlap	Keeps context continuity across splits
Document metadata	Helps identify which source doc answered the question
Low-rank adaptation (LoRA)	Fine-tune LLM on domain knowledge for better answers

Summary / Key Takeaways

1. RAG adds **retrieval context** to LLM answers
2. Steps: **Documents → Split → Embed → Store → Retrieve → Generate**
3. Works **locally** if you run both the **LLM and vector store** on your machine
4. Provides **private, accurate, and domain-specific responses**

Theory 4: Practical Setup & Code Flow

Required Libraries

Library	Purpose
transformers	Load local LLM models (e.g., LLaMA, Vicuna, MPT)
torch	PyTorch backend for model computation
sentence-transformers	Generate embeddings for retrieval
langchain	Orchestrate RAG pipeline (retriever + LLM)
faiss-cpu / chromadb	Vector database for fast similarity search
PyPDF2 / pdfplumber	Extract text from PDFs

Minimal Data Flow

1. **Load documents** → PDF / txt → raw text
2. **Split into chunks** → ~300–500 tokens each
3. **Generate embeddings** → numeric vector for each chunk
4. **Store in vector DB** → FAISS or Chroma
5. **Query → Embed → Retrieve top-k** → LLM generates answer

Code Flow (Minimal Python Example)

✅ This is the **core working pipeline**:

- Local LLM generates text
- FAISS retrieves context
- LangChain orchestrates retrieval + generation

```
# 1. Import libraries

from langchain.vectorstores import FAISS

from langchain.embeddings import SentenceTransformerEmbeddings

from langchain.chains import RetrievalQA

from langchain.llms import HuggingFacePipeline

from transformers import AutoTokenizer, AutoModelForCausalLM, pipeline


# 2. Load local LLM

model_name = "TheBloke/vicuna-7B-1.1-HF"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(model_name, device_map="auto", torch_dtype="auto")


pipe = pipeline("text-generation", model=model, tokenizer=tokenizer, max_length=512)

llm = HuggingFacePipeline(pipeline=pipe)


# 3. Load embeddings

embeddings = SentenceTransformerEmbeddings(model_name="all-MiniLM-L6-v2")


# 4. Load / create vector store (FAISS)

# Assume we already split docs and have text chunks

vector_store = FAISS.load_local("my_faiss_index", embeddings)


# 5. Build RetrievalQA chain

qa = RetrievalQA.from_chain_type(llm=llm, retriever=vector_store.as_retriever())


# 6. Ask a question

query = "Explain attention mechanism in simple words"

answer = qa.run(query)

print(answer)
```

Practical Notes

- **Document splitting**
  - Use langchain.text_splitter.CharacterTextSplitter
  - Chunk size ~300–500 tokens, overlap ~50 tokens
- **Vector store**
  - Save after building to avoid recomputation:
- vector_store.save_local("my_faiss_index")
- **Querying**
  - Embed the query → retrieve top-k → pass to LLM
  - Top-k = 3–5 is usually enough
- **Hardware Tips**
  - 7B model → GPU with 12–16GB VRAM
  - CPU → smaller models / quantized versions
  -

Key Takeaways

- RAG allows **LLM to access your own documents**
- LangChain simplifies the pipeline **retriever + generator**
- Minimal Python code is enough to **get a working system**
- This is **the foundation** before you move to fine-tuning or more complex setups

Theory 5: Fine-Tuning & Enhancements

Once your basic **Local LLM + RAG** system is running, you can enhance it for better performance, accuracy, and usability.

Why Fine-Tune / Enhance?

Even with RAG:

- LLM may not fully understand your **domain-specific style**
- It might still **hallucinate slightly**
- Prompts may not be **optimized**
- Performance may be slow for **large datasets**

Enhancements solve these problems.

Fine-Tuning LLM (Optional but Powerful)

Goal: Adapt your LLM to your **specific domain or style**.

Methods:

1. **LoRA (Low-Rank Adaptation)**
   - Fine-tunes only a small portion of the model weights
   - Memory-efficient → can fine-tune 7B+ models on 12–16GB GPU
   - Example use:
     - Fine-tune Vicuna on your **internal notes**
     - System gives answers in your style
2. **Full Fine-Tuning**
   - Changes all model weights
   - Requires huge GPU + time
   - Usually unnecessary for RAG
3. **Instruction Tuning**
   - Train the model to **follow your prompts better**
   - Example: Train on Q&A pairs from your notes

Prompt Engineering

Even without fine-tuning, **prompts make a big difference**.

- Include **retrieved chunks in prompt**
- Example:

Context: [retrieved chunks]

Instruction: Answer the question using ONLY the context above.

Question: [user query]

Answer:

- Helps **reduce hallucination**
- Helps **focus LLM on relevant info**
-

Vector Store Optimizations

- **Top-k tuning** → Usually 3–5 chunks is enough
- **Chunk size** → ~300–500 tokens
- **Metadata** → Store PDF name, page, section for **traceable answers**
- **Embeddings** → Use domain-specific embeddings for better retrieval
-

Speed & Memory Enhancements

- **Quantization** → Reduce model from FP16 → 8-bit / 4-bit
  - Faster inference, smaller memory footprint
- **GPU Offloading** → Offload some layers to CPU
- **Batch embedding** → Embed documents in batches to save time
- **Caching** → Store retrieved results for common queries

6 Advanced Features

- **Multimodal RAG** → Combine text + images
- **Chained RAG** → Multiple retrievers (PDF + Web) for a richer context
- **Active Learning** → Add feedback loop to improve embeddings or LLM answers
-

7 Summary / Key Takeaways

1. Fine-tuning improves **domain-specific performance**
2. LoRA is **best for local models** (memory-efficient)
3. Prompt engineering can significantly reduce **hallucinations**
4. Optimizing embeddings, chunking, and vector stores improves **accuracy & speed**
5. Advanced RAG setups can integrate **multimodal sources**

-Parichay Sengupta